Regresión regularizada

Sesión 5 · Módulo 2: Modelos de regresión

Curvas de aprendizaje · Ridge (L2) · Lasso (L1) · Elastic Net · Alta dimensión

Datos simulados de alta dimensión · scikit-learn

Plan de la sesión (≈45 min)

Qué veremos

- **A.** Sobreajuste: revisión con curvas de aprendizaje
- **B.** Regresión Ridge (L2): intuición y efecto sobre los coeficientes
- **C.** Regresión Lasso (L1): dispersión y selección de variables
- **D.** Elastic Net: equilibrio entre L1 y L2; ajuste de hiperparámetros
- **E.** Comparación de OLS, Ridge y Lasso en un conjunto de **alta dimensión**

Una sola idea de fondo: **cobrar un "impuesto" a los pesos grandes** para que el modelo no memorice el ruido.

A1 · De dónde venimos: la flexibilidad cuesta

Lo que vimos en la Sesión 4

- Un modelo muy flexible (polinomio de grado alto) **ajusta de maravilla** los datos de entrenamiento...
- ...pero su error en datos **nuevos** sube: **sobreajuste** (varianza alta).
- Cada columna nueva agrega **un peso más** que aprender.

Hoy: muchas variables, pocos datos

Con 150 variables y solo 100 observaciones hay **más incógnitas que datos**. Ya no basta con elegir el grado: necesitamos una forma de **frenar** al modelo.

Dónde pasa en la vida real

- **Genómica**: miles de genes, decenas de pacientes.
- **Texto**: miles de palabras como variables.
- **Ingeniería de variables**: polinomios e interacciones (329 columnas con grado 4).
- **Sensores / imágenes**: muchas mediciones por muestra.

Llamamos **alta dimensión** al caso en que el número de variables p es comparable o mayor que el de observaciones n.

A2 · Curvas de aprendizaje: ¿qué son y cómo se leen?

Cómo se construyen

1. Entrenamos el mismo modelo con **más y más datos** (n crece).
2. Para cada n medimos el error en el **entrenamiento** y en **datos no vistos**.
3. Graficamos ambos errores contra n.

La pregunta que responden

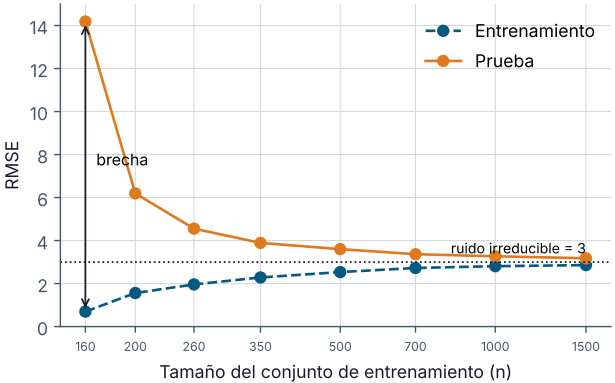
¿Mi problema es el **modelo** o son los **datos**? ¿Me ayudaría conseguir más datos?

Cómo se leen

	Las curvas...	**Diagnóstico**
Subajuste	ambas **altas** y juntas	modelo muy simple
Bien	ambas bajas y cercanas	todo en orden
Sobreajuste	entrenamiento bajo, prueba alta: **brecha**	modelo muy flexible

Hoy buscamos **cerrar la brecha** sin necesitar más datos.

A3 · Curva de aprendizaje de OLS con 150 variables



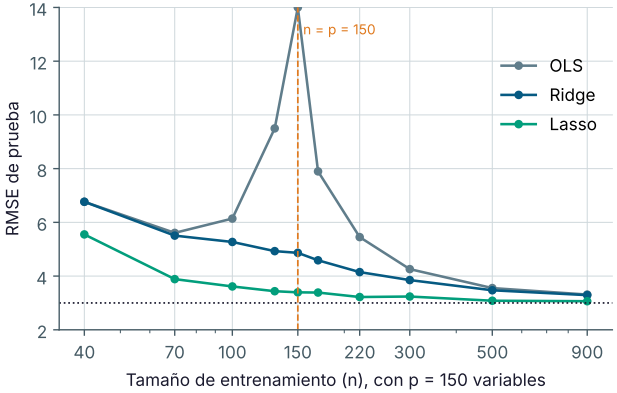
Lectura

- Con $n = 160$ (apenas más que las 150 variables): error de **entrenamiento 0.70**, de **prueba 14.19**. ¡Memoriza!
- Al crecer n la brecha se **cierra**: con $n = 1500$ son 2.86 y 3.18.
- El error de prueba tiende al **ruido irreducible** (3): no se puede bajar de ahí.

Moraleja

Más datos curan el sobreajuste, pero **no siempre los tenemos**.

A4 · Cuando n no crece: OLS se descontrola cerca de $n = p$



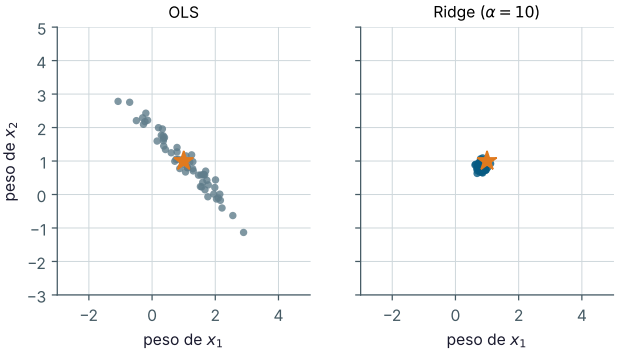
Lectura

- **OLS** (gris): con $n = p = 150$ el error de prueba llega a 47.4 (la gráfica lo corta en 14). Con $n = 100$ es 6.14.
- **Ridge** y **Lasso** (hoy los vemos) siguen estables aun con $n < p$: a $n = 100$ dan 5.27 y 3.61.
- Con muchísimos datos ($n = 900$) los tres se parecen.

Idea

Regularizar vale más justo cuando n es chico respecto a p.

A5 · Otra señal de sobreajuste: pesos inestables



El experimento

Dos variables casi idénticas ($\rho = 0.97$) y $y = x_1 + x_2 +$ ruido. Repetimos 60 veces con **otra muestra** de 30 puntos y anotamos los pesos. La estrella es la verdad $(1, 1)$.

Lectura

- **OLS**: los pesos saltan de una muestra a otra (desviación 0.85 en w_1), aunque la *suma* se mantenga: el modelo no sabe a quién culpar.
- **Ridge**: nube compacta (desviación 0.10).

A6 · La idea: cobrar un “impuesto” a los pesos grandes

Pérdida regularizada

$$\underbrace{\text{Error de ajuste}}_{\text{lo que ya conocemos}} + \alpha \cdot \underbrace{\text{Penalización(pesos)}}_{\text{castigo a pesos grandes}}$$

El modelo ya no busca solo “ajustar bien”: busca **ajustar bien con pesos pequeños**.

El dial $\alpha \geq 0$

- $\alpha = 0$: sin impuesto $\Rightarrow$ OLS (riesgo de **varianza** alta).
- α muy grande: pesos $\approx 0 \Rightarrow$ modelo casi constante (**sesgo** alto).
- Se elige con **validación cruzada**.

Dos formas de medir “grande”

	Penaliza	Nombre
L2	suma de w_j^2	**Ridge**
L1	suma de $\lvert w_j \rvert$	**Lasso**
Mezcla	ambas	**Elastic Net**

Es el equilibrio sesgo–varianza

Aceptamos **un poco de sesgo** a cambio de **bajar mucho la varianza**. Es el mismo trato de la Sesión 4, ahora con un dial continuo.

Pausa para pensar (actividad 1, 2 min)

Trabaja de forma individual

Un modelo con $n = 100$ datos obtiene RMSE de **0.5** en entrenamiento y **2.4** en validación.

1. ¿Es subajuste o sobreajuste? ¿Cómo lo sabes?
2. Si dibujas la curva de aprendizaje, ¿qué esperas que pase con la brecha al aumentar n?
3. No puedes conseguir más datos. ¿Qué harías con los pesos del modelo?

Compara después con un compañero. Respuestas: (1) sobreajuste: entrenamiento muy bajo y brecha grande; (2) la brecha se cierra poco a poco, con ambas curvas acercándose; (3) penalizar los pesos grandes (regularizar) para bajar la varianza.

B1 · Ridge (L2): sumar el cuadrado de los pesos

Objetivo de Ridge

$$\min_{w} \sum_{i} \left(y_i - \hat{y}_i\right)^2 + \alpha \sum_{j=1}^{p} w_j^2$$

Error al cuadrado de siempre, más α veces la **suma de pesos al cuadrado**. El intercepto no se penaliza.

Intuición

Elevar al cuadrado hace que **un peso grande cueste muchísimo** y uno pequeño casi nada. El modelo prefiere **repartir** el trabajo entre muchas variables en vez de depender de unas pocas con pesos enormes.

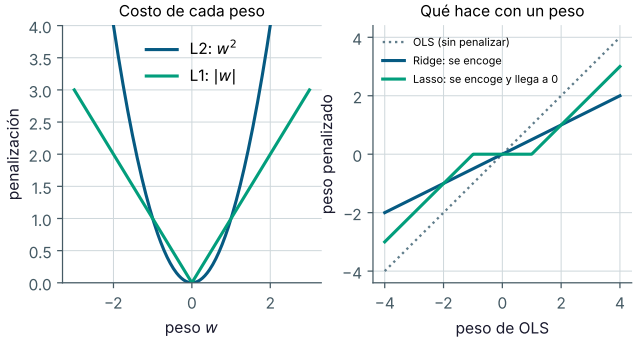
Propiedades

- Sigue teniendo **solución cerrada**: $(X^\top X + \alpha I)^{-1} X^\top y$.
- Sumar αI vuelve la matriz **invertible** aun con $p > n$ o variables repetidas.
- Encoge todos los pesos, pero **ninguno llega exactamente a 0**.

Dos variables casi iguales

Si $x_1 \approx x_2$ y ambas ayudan, OLS puede dar $(+50, -48)$; Ridge prefiere $(1, 1)$, porque $1^2 + 1^2$ cuesta mucho menos que $50^2 + 48^2$.

B2 · ¿Qué hace con un peso? Lo encoge proporcionalmente



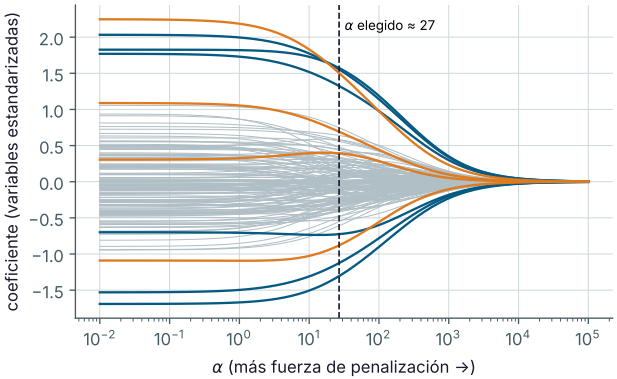
Izquierda: el costo

La parábola w^2 es **casi plana cerca de 0** y se dispara lejos: perdona lo pequeño y castiga lo grande. (El valor absoluto, en verde, lo veremos en Lasso.)

Derecha: el efecto

Para cada peso de OLS (eje horizontal), la línea azul muestra el peso de Ridge (con $\alpha = 1$): **siempre una fracción** del original. Se acerca a 0 pero **no lo toca**.

B3 · Efecto sobre los coeficientes: la trayectoria de Ridge



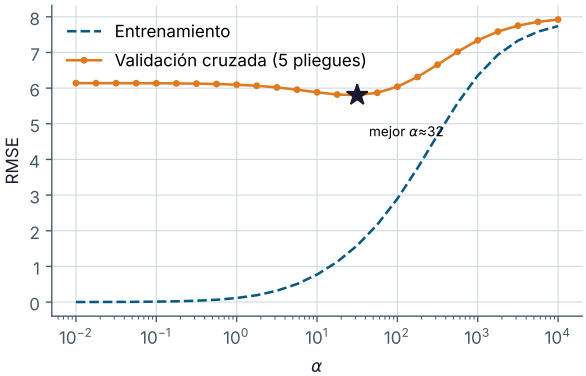
Cómo leerla

Cada curva es **el peso de una variable** al subir α (hacia la derecha). Azul y naranja: las 10 variables que sí importan. Gris: las 140 de puro ruido.

Lectura

- Con α pequeño los pesos son **grandes y desordenados**.
- Al subir α **todos se contraen suavemente** hacia 0.
- Ninguna curva toca el cero de golpe: Ridge **no elimina variables**.

B4 · Elegir α: validación cruzada



Lectura

- **Entrenamiento** (azul): sube con α; ajustar siempre cuesta más cuando frenamos al modelo.
- **Validación** (naranja): baja, llega a un mínimo ($\alpha \approx 32$, RMSE 5.80) y vuelve a subir: **subajuste**.
- Es la misma forma de "U" de la Sesión 4, pero con α en el eje: **más α = menos complejidad**.

Receta

Probar α en una **escala logarítmica** (de 10^{-2} a 10^{4}) y quedarse con el que minimiza el error de validación.

B5 · En scikit-learn: `RidgeCV` dentro de un `Pipeline`

```python
import numpy as np
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import RidgeCV
from sklearn.model_selection import KFold

kf = KFold(5, shuffle=True, random_state=0)

ridge = make_pipeline(
    StandardScaler(),
    RidgeCV(alphas=np.logspace(-2, 4, 50), cv=kf))
ridge.fit(Xtr, ytr)

print(ridge[-1].alpha_)        # alpha elegido
print(ridge[-1].coef_[:5])     # primeros pesos
```

Puntos clave

- **Estandarizar siempre**: la penalización trata todos los pesos igual; una variable en milímetros tendría un peso 1000 veces menor que en metros y casi no se castigaría.
- El escalador va **dentro del pipeline**: no hay fuga de datos en la validación.
- `RidgeCV` prueba la lista de `alphas` y se queda con el mejor.
- `Ridge(alpha=...)` si ya conoces el valor.

Pausa para pensar (actividad 2, 3 min)

Trabaja de forma individual

1. ¿A qué modelo equivale Ridge con $\alpha = 0$? ¿Y qué pasa con los pesos cuando $\alpha \to \infty$?
2. Tienes dos columnas **idénticas**, $x_1 = x_2$, y los datos cumplen $y = 2x_1$. ¿Qué pesos (w_1, w_2) encuentra Ridge? ¿Por qué OLS no puede decidirse?
3. ¿Por qué hay que estandarizar antes de usar Ridge?

Respuestas: (1) OLS; todos los pesos $\to 0$ y el modelo predice el promedio. (2) Ridge da $(1, 1)$: $1^2 + 1^2 = 2$ cuesta menos que $(2, 0)$ con 4; OLS acepta cualquier (w_1, w_2) con $w_1 + w_2 = 2$. (3) Para que la penalización no dependa de las unidades de cada variable.

C1 · Lasso (L1): sumar el valor absoluto de los pesos

Objetivo de Lasso

$$\min_{w} \frac{1}{2n} \sum_{i} \left(y_i - \hat{y}_i\right)^2 + \alpha \sum_{j=1}^{p} |w_j|$$

Cambiamos el cuadrado por el **valor absoluto**. (*Least Absolute Shrinkage and Selection Operator.*)

Propiedades

- Encoge los pesos **y** lleva varios **exactamente a cero**.
- Un peso en cero significa que la variable **sale del modelo**: selección automática.
- No tiene fórmula cerrada: se resuelve con algoritmos iterativos (**descenso por coordenadas**).

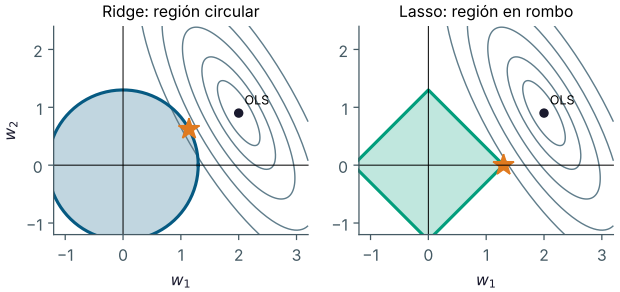
Intuición

El valor absoluto cobra **lo mismo por cada unidad** de peso, sea grande o pequeña. Una variable que casi no ayuda no "paga su impuesto" y el modelo prefiere **apagarla** (peso 0).

Resultado: un modelo disperso

Dispersión (*sparsity*): de las p variables, solo unas pocas tienen peso distinto de cero. El modelo es más **corto** y **fácil de explicar**.

C2 · ¿Por qué Lasso llega a cero? La geometría



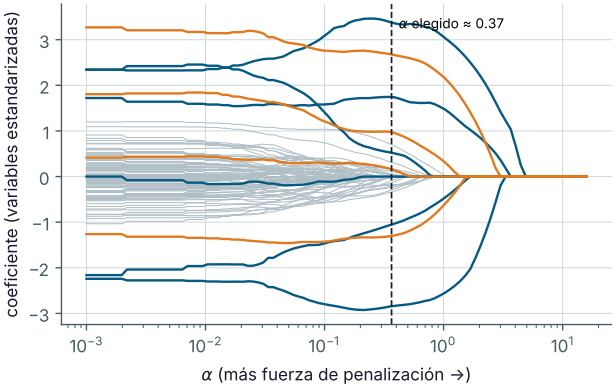
Cómo leerla

Con dos pesos (w_1, w_2): las elipses grises son los **niveles de error** (centro = OLS). La región coloreada es el **presupuesto** de pesos permitido. La estrella es la solución: el punto de la región más cercano al centro.

Lectura

- **Círculo (Ridge)**: no tiene esquinas; el contacto cae en cualquier lado y **ambos pesos quedan distintos de cero**.
- **Rombo (Lasso)**: tiene **esquinas sobre los ejes**; el contacto suele ocurrir ahí y un peso vale **exactamente 0**.

C3 · La trayectoria de Lasso: las variables van saliendo



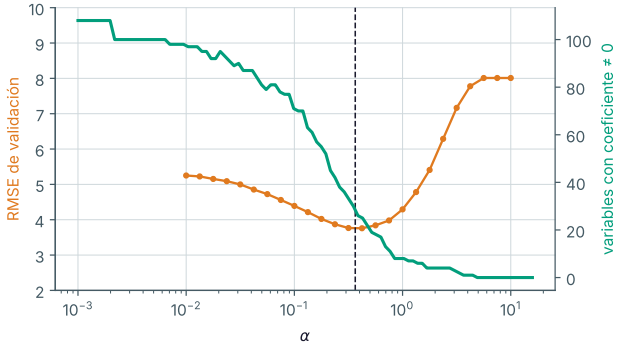
Cómo leerla

Igual que la de Ridge: cada curva es una variable y α crece hacia la derecha. **Compara con la de Ridge**: aquí las curvas **se pegan al cero y se quedan**.

Lectura

- Con α muy grande **todos** los pesos valen 0.
- Al bajar α las variables **van entrando una por una**; las más útiles (azul, naranja) entran primero.
- Las 140 variables de ruido (gris) entran tarde, con pesos pequeños.

C4 · ¿Cuántas variables quedan? Dispersión frente a error



Lectura

- **Verde**: número de variables con peso $\neq 0$; baja al subir α.
- **Naranja**: error de validación; mínimo en $\alpha \approx 0.37$, donde Lasso conserva **30 de 150** variables.
- Con más α se queda con menos variables, pero el error **vuelve a subir**: se apagan variables útiles.

Dos objetivos distintos

El α de **mejor predicción** suele conservar **más** variables de las necesarias. Si quieres un modelo más corto, puedes elegir un α algo mayor.

C5 · Lasso como selección de variables: ventajas y límites

Lo que sí hace bien

- **Selección integrada** en el entrenamiento: método *embedded* de la Sesión 3.
- Modelo **interpretable**: solo hay que explicar las variables con peso distinto de cero.
- Funciona con $p > n$.
- En nuestro experimento recupera **9.8 de las 10** variables reales (promedio de 20 réplicas).

Sus límites

- **Falsos positivos**: conserva en promedio 29 variables, es decir, **unas 20 variables de ruido** se cuelan.
- **Variables correlacionadas**: elige **una** del grupo y descarta las demás, y la elegida puede cambiar con la muestra.
- **Sesgo**: también encoge los pesos de las variables que sí importan.
- Con $p > n$ conserva **a lo más** n variables.

C6 · En scikit-learn: LassoCV

```python
from sklearn.linear_model import LassoCV

lasso = make_pipeline(
    StandardScaler(),
    LassoCV(alphas=np.logspace(-3, 1.2, 60),
            cv=kf, max_iter=100000))
lasso.fit(Xtr, ytr)

coef = lasso[-1].coef_
print(lasso[-1].alpha_)         # alpha elegido
print((coef != 0).sum())        # variables que quedan
print(np.flatnonzero(coef))     # cuales son
```

Puntos clave

- `coef_ != 0` es la **selección de variables**.
- Estandarizar también es obligatorio.
- `max_iter` alto: el algoritmo es iterativo y con α pequeño tarda en converger.
- Con una lista de `alphas` explícita el código funciona igual en todas las versiones de scikit-learn.

Notación

El α de Lasso y el de Ridge **no son comparables** (Lasso divide el error entre $2n$). Cada uno se ajusta por separado.

Pausa para pensar (actividad 3, 3 min)

¿Verdadero o falso? Justifica en una frase

(a) Con $\alpha = 0$, Lasso y Ridge dan los mismos pesos que OLS.

(b) Ridge suele dejar varios pesos exactamente en cero.

(c) Subir α aumenta el **sesgo** y reduce la **varianza**.

(d) Si queremos **interpretar** qué variables importan, Lasso es más natural que Ridge.

Respuestas: (a) V, sin penalización los tres son el mismo problema. (b) F, Ridge encoge pero casi nunca llega a cero. (c) V, es el equilibrio sesgo–varianza. (d) V, porque descarta variables.

D1 · Elastic Net: una mezcla de L1 y L2

Objetivo de Elastic Net

$$\min_{w} \frac{1}{2n}\sum_{i}(y_i - \hat{y}_i)^2 + \alpha\left[\rho\sum_{j}|w_j| + \frac{1-\rho}{2}\sum_{j}w_j^2\right]$$

Dos hiperparámetros: α (**cuánto** penalizar) y ρ (**qué mezcla**; en scikit-learn, `l1_ratio`).

El dial ρ (`l1_ratio`)

$\rho = 1$	todo L1: **Lasso**
$0 < \rho < 1$	mezcla: **Elastic Net**
$\rho \to 0$	casi todo L2: **Ridge**

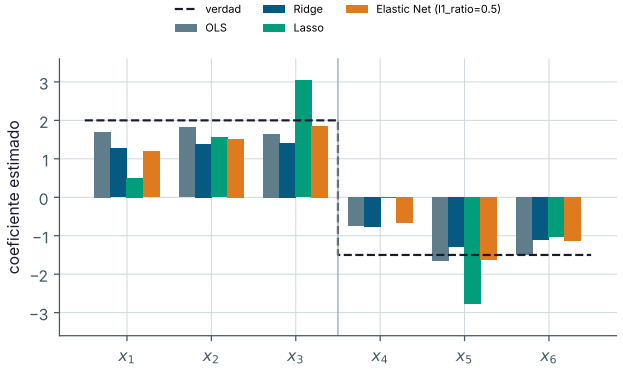
Qué buscamos

Quedarnos con lo **mejor de cada uno**: el **cero** de Lasso (selección) y la **estabilidad** de Ridge (reparte el peso entre variables parecidas).

Cuándo usarlo

- Hay **grupos de variables correlacionadas**.
- $p > n$ y queremos más de n variables posibles.
- No sabemos si conviene Ridge o Lasso: dejamos que la validación decida ρ.

D2 · Variables correlacionadas: qué hace cada método



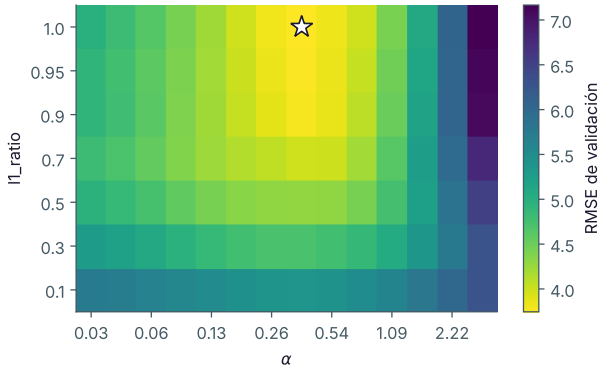
El caso

x_1, x_2, x_3 son **tres copias ruidosas** de la misma señal (correlación ≈ 0.8) y las tres valen $+2$; igual con x_4, x_5, x_6 (valen -1.5). La línea punteada es la verdad.

Lectura

OLS y **Ridge** dan pesos parecidos dentro de cada grupo (los de Ridge, más pequeños). **Lasso** reparte **desigual**: en el grupo 1 pone 0.5, 1.6 y 3.0, y en el grupo 2 **descarta** x_4. **Elastic Net** con `l1_ratio`=0.5 queda en medio: pesos más parejos (1.2, 1.5, 1.9) y conserva x_4. Cuesta más variables: 64 frente a 29 de Lasso. (Una sola réplica; la validación cruzada eligió aquí 0.95.)

D3 · Dos hiperparámetros: α y `l1_ratio`



Búsqueda en malla

Cada celda es una pareja (α, `l1_ratio`) evaluada con validación cruzada. Más oscuro = menor error. La estrella marca la mejor: `l1_ratio`=1 (es decir, Lasso puro), $\alpha \approx 0.38$, RMSE 3.75.

Lectura

- Para cada razón hay un α **óptimo**: el valle atraviesa la figura.
- Aquí la verdad es dispersa, así que gana una razón cercana a 1; la fila de 0.1 (casi Ridge) solo llega a 5.38. Con otros datos el valle se mueve hacia abajo.
- Los dos hiperparámetros **interactúan**: hay que buscarlos **juntos**.

D4 · Ajuste de hiperparámetros: ElasticNetCV y GridSearchCV

```python
from sklearn.linear_model import ElasticNetCV, ElasticNet
from sklearn.model_selection import GridSearchCV

# Opcion 1: el atajo (busca alpha y l1_ratio)
en = make_pipeline(StandardScaler(),
    ElasticNetCV(l1_ratio=[.1, .3, .5, .7, .9, .95],
                 alphas=np.logspace(-3, 1.2, 60),
                 cv=kf, max_iter=100000))
en.fit(Xtr, ytr)
print(en[-1].alpha_, en[-1].l1_ratio_)

# Opcion 2: la malla explicita
pipe = make_pipeline(StandardScaler(),
                     ElasticNet(max_iter=100000))
malla = {"elasticnet__alpha": np.logspace(-1.5, .5, 14),
         "elasticnet__l1_ratio": [.1, .3, .5, .7, .9,
              .95, 1.]}
bus = GridSearchCV(pipe, malla, cv=kf,
        scoring="neg_root_mean_squared_error")
bus.fit(Xtr, ytr)
print(bus.best_params_)
```

Puntos clave

- `ElasticNetCV`: rápido, especializado.
- `GridSearchCV`: general, sirve para **cualquier** modelo (lo retomaremos en la sesión de evaluación).
- En el pipeline los parámetros se nombran `paso__parametro`.
- El conjunto de **prueba** no se toca durante la búsqueda.

Costo

14×7 parejas $\times$ 5 particiones $= 490$ ajustes. Con pocos hiperparámetros es barato; la malla crece **multiplicativamente**.

D5 · ¿Cuándo usar cada uno?

	OLS	**Ridge (L2)**	**Lasso (L1)**	**Elastic Net**
Penalización	ninguna	$\sum w_j^2$	$\sum \lvert w_j \rvert$	mezcla de ambas
Pesos en cero	no	no (se acercan)	**sí**	sí, menos que Lasso
Variables correlacionadas	inestable	**reparte el peso**	elige una	**reparte el peso**
Hiperparámetros	0	α	α	α y `l1_ratio`
Úsalo cuando...	$n \gg p$ y pocas variables	todas las variables aportan un poco	se espera que **pocas** variables importen	hay grupos correlacionados o dudas entre ambos

En todos los casos: **estandarizar** las variables y elegir los hiperparámetros con **validación cruzada**.

E1 · El experimento: un conjunto de alta dimensión

Los datos (simulados, con verdad conocida)

- $n = 100$ observaciones de entrenamiento, $p = 150$ variables ($p > n$). Prueba: 3 000 observaciones nuevas.
- Solo **10 variables importan** ($x_1, \ldots, x_{10}$); las otras 140 son ruido puro.
- x_1–x_3 y x_4–x_6 son grupos **correlacionados** (≈ 0.8).
- Ruido de las observaciones: $\sigma = 3 \Rightarrow$ ningún modelo puede bajar de **RMSE = 3**.

Los cuatro modelos

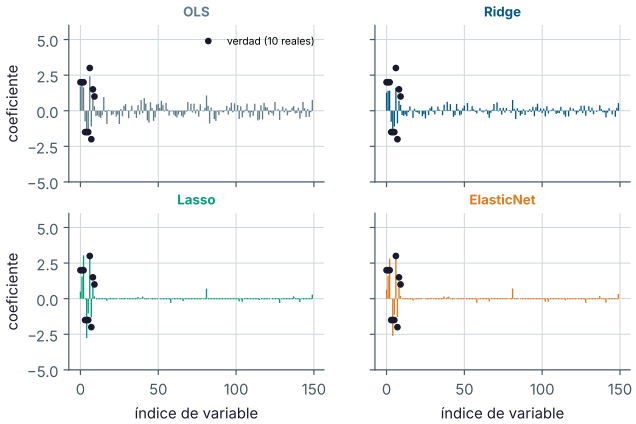
Todos con `StandardScaler` en el pipeline y los hiperparámetros elegidos por validación cruzada de 5 particiones:

1. **OLS** (`LinearRegression`; con $p > n$ toma la solución de menor norma).
2. **Ridge** (`RidgeCV`).
3. **Lasso** (`LassoCV`).
4. **Elastic Net** (`ElasticNetCV`).

Por qué simulado

Como **conocemos la verdad**, podemos medir si cada método recupera las variables correctas. Para que no dependa de una sola muestra, repetimos 20 veces con datos nuevos.

E2 · Coeficientes: qué estima cada método



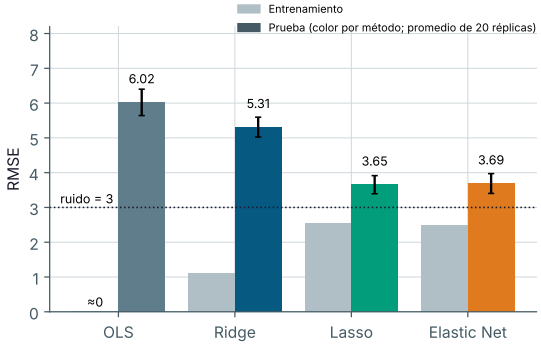
Cómo leerla

Cada barra es el peso de una variable (eje horizontal: las 150). Los puntos negros son los **10 valores verdaderos**; todo lo demás debería ser cero.

Lectura

- **OLS**: pesos grandes y desordenados en **todas** las variables.
- **Ridge**: pesos más chicos, pero **las 150 siguen activas**.
- **Lasso** y **Elastic Net**: la mayoría en cero; las barras caen cerca de los puntos verdaderos.

E3 · Error de entrenamiento y de prueba



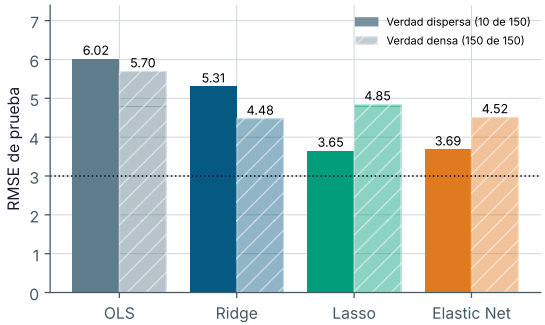
Promedio de 20 réplicas

	RMSE entren.	RMSE prueba	Variables usadas	Reales halladas
OLS	0.00	6.02	150	10.0
Ridge	1.11	5.31	150	10.0
Lasso	2.54	3.65	29	9.8
Elastic Net	2.49	3.69	33	9.9

Lectura

OLS memoriza (entrenamiento ≈ 0) y generaliza mal. **Ridge** mejora un poco. **Lasso** y **Elastic Net** se acercan al piso de 3 porque **la verdad es dispersa**. Entre ellos hay una diferencia mínima, menor que la variación entre réplicas.

E4 · ¿Y si todos los coeficientes importan un poco?



El experimento

Misma construcción, pero ahora **los 150 coeficientes** son distintos de cero y pequeños (verdad **densa**). Mismo $n = 100$ y 20 réplicas.

Lectura

- Con verdad **dispersa**: gana Lasso / Elastic Net.
- Con verdad **densa**: **Ridge** y Elastic Net quedan mejor; **Lasso** pierde porque apaga variables que sí aportaban.
- **Elastic Net** queda cerca del mejor en **ambos** escenarios.

E5 · Conclusiones de la comparación

Lo que vimos

- En alta dimensión, **OLS sobreajusta**: error de entrenamiento casi 0, de prueba alto.
- **Regularizar** baja la varianza a cambio de un poco de sesgo.
- **Ridge**: encoge, estabiliza; no selecciona.
- **Lasso**: encoge y selecciona; brilla si pocas variables importan.
- **Elastic Net**: opción segura cuando no sabemos cómo es la verdad.

No hay método ganador universal

El mejor método depende de **cómo es el problema** (¿pocas variables importantes o muchas pequeñas?). Se decide **con validación**, no por costumbre.

Precauciones

- Esto es una **simulación**: en datos reales no conocemos la verdad.
- Lasso incluye **falsos positivos**: "seleccionada" no significa "causal".
- Siempre estandarizar y reservar un conjunto de **prueba** intacto.

Resumen de la sesión

Ideas clave

- **Curvas de aprendizaje**: la brecha entre entrenamiento y validación delata el sobreajuste; más datos la cierran, pero a veces no los hay.
- **Regularización**: cobrar un impuesto α·penalización a los pesos grandes. $\alpha = 0$ es OLS.
- **Ridge (L2)**: encoge todos los pesos, estabiliza con variables correlacionadas, no los lleva a 0.

Para llevarse

- **Lasso (L1)**: lleva pesos a 0, así que **selecciona variables**; ojo con correlacionadas y falsos positivos.
- **Elastic Net**: mezcla ambas con `l1_ratio`; se ajusta con `ElasticNetCV` o `GridSearchCV`.
- Siempre: **estandarizar** dentro del `Pipeline`, elegir α por **validación cruzada** y evaluar en prueba **una sola vez**.

Siguiente sesión

Regresión con máquinas de vectores de soporte (SVR): otra forma de controlar la complejidad.

Actividad de laboratorio (individual)

Comparación de OLS, Ridge y Lasso en un conjunto de datos de alta dimensión

Cuaderno de Jupyter con datos simulados ($n = 100$, $p = 150$). Trabaja **de forma individual**.

1. Genera los datos, divide en entrenamiento y prueba y ajusta **OLS**: compara el error de entrenamiento con el de prueba. ¿Hay sobreajuste?
2. Dibuja la **curva de aprendizaje** de OLS con n creciente (tamaños de 40 a 900).
3. Ajusta **Ridge** con varios α; grafica la **trayectoria de los coeficientes** y el error de validación.
4. Ajusta **Lasso**; grafica la trayectoria y cuenta cuántas variables quedan para distintos α.
5. Ajusta **Elastic Net** con `GridSearchCV` sobre α y `l1_ratio`; muestra el mapa de calor del error.
6. Compara los cuatro modelos en prueba (RMSE, variables usadas, variables reales halladas) y grafica sus coeficientes.
7. Repite con una verdad **densa**. ¿Cambia el mejor método? Explica por qué.

Entregable: el cuaderno con tus respuestas a preguntas de interpretación (¿cuándo conviene Ridge frente a Lasso?, ¿qué `l1_ratio` elegiste y por qué?).



¿Preguntas?

¡Pasemos a la actividad: comparar OLS, Ridge y Lasso en alta dimensión!